

# Rencana Kerja Tim UAS Sistem Terdistribusi

Project: Ekstensi (`goshlive/dist-system`) Case Study: **Appointment Booking + Notification**  
Tim: 3 orang Target: nilai maksimal (100%)

---

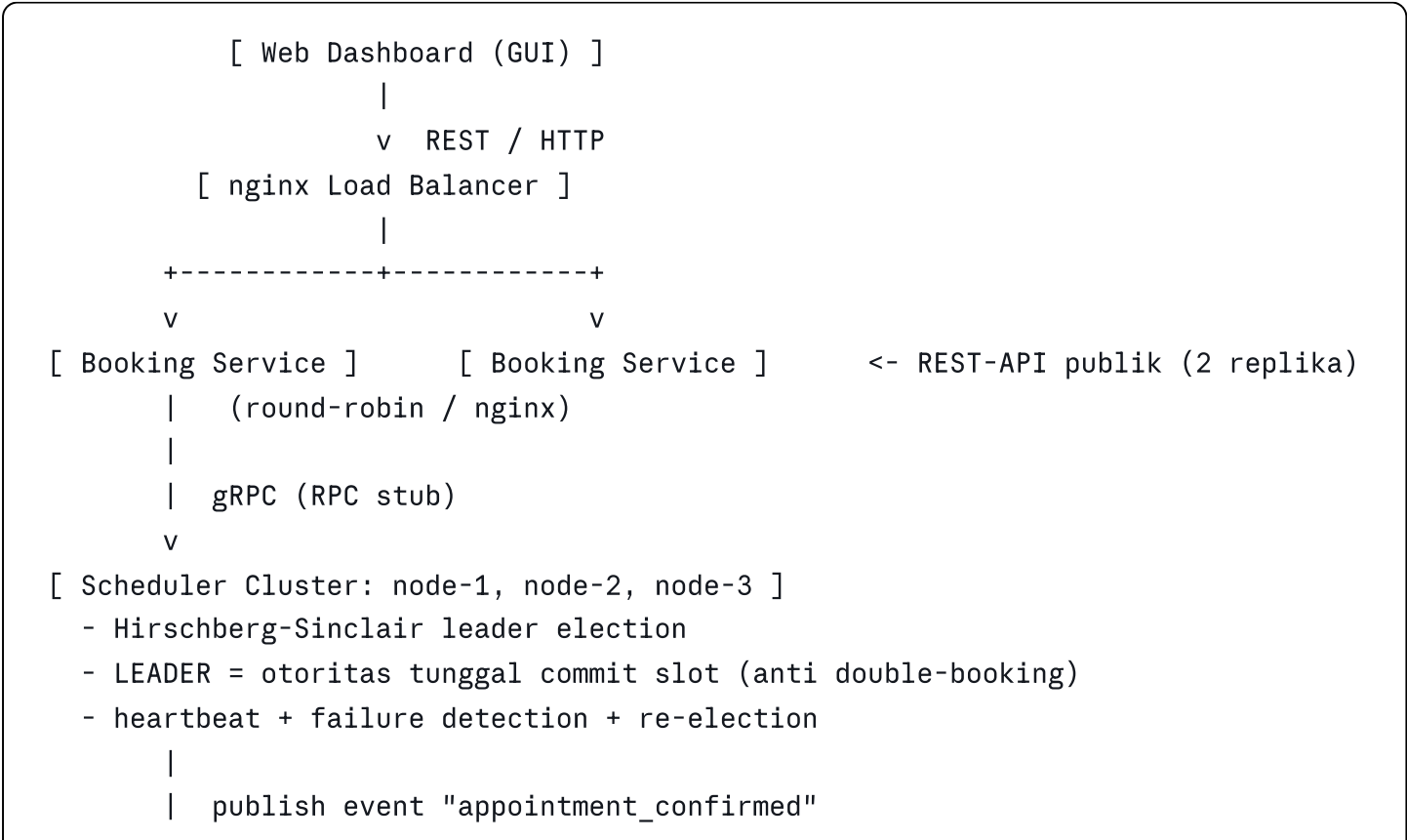
## 1. Ringkasan Strategi

Kita ambil (`payment-system/`) di repo demo sebagai fondasi, lalu kita ubah jadi sistem booking janji temu yang lengkap. Logika "leader sebagai otoritas tunggal" yang di demo dipakai buat proses pembayaran, kita pakai ulang buat **alokasi slot booking** supaya gak ada double-booking di slot/jam yang sama. Ini alasan teknis yang kuat kenapa kita butuh leader election, dan bakal jadi poin bagus pas presentasi.

Keputusan kunci buat ngejar nilai maksimal:

- 1. **Hirschberg-Sinclair (HS)** buat leader election (20%, bukan Ring 10% atau Bully 5%).
  - 2. **gRPC stub** buat komunikasi internal antar service (biar beda jelas dari REST-API publik).
  - 3. **PostgreSQL** sebagai persistent storage (ganti in-memory dict di demo).
  - 4. **nginx** sebagai load balancer di depan 2 replika Booking Service.
  - 5. **Web dashboard** sebagai GUI buat demo sekaligus nilai GUI.
- 

## 2. Arsitektur Target





Alur happy path:

- 1. User submit booking lewat dashboard ke Booking Service (via nginx).
- 2. Booking Service panggil Scheduler leader via gRPC buat reserve slot.
- 3. Leader cek slot di DB, commit kalau available, balikin konfirmasi.
- 4. Booking Service simpan appointment ke DB, publish event ke RabbitMQ.
- 5. Notification Worker konsumsi event, kirim notifikasi (simulasi), tulis log ke DB.
- 6. Dashboard nampilin status appointment + status leader + log notifikasi real-time.

Skenario failover (buat demo): matiin node leader, dalam beberapa detik HS milih leader baru, booking tetap jalan.

### 3. Pemetaan Rubrik ke Fitur

| Item rubrik                    | Bobot                    | Implementasi kita                  | Pemilik |
|--------------------------------|--------------------------|------------------------------------|---------|
| REST-API                       | 15%                      | Booking Service (FastAPI/Flask)    | B       |
| Service-to-Service RPC         | 20%                      | gRPC stub: Booking <-> Scheduler   | B + A   |
| RabbitMQ async workflow        | 20%                      | Pipeline notifikasi                | B       |
| Persistent Storage (DB)        | 10%                      | PostgreSQL                         | C       |
| Case study domain beda         | 10%                      | Appointment booking + notification | semua   |
| Leader Election beda algoritma | 20%                      | Hirschberg-Sinclair                | A       |
| API Load Balancing             | 5%                       | nginx (2 replika Booking)          | C       |
| Subtotal Bagian A              | 100% (= 50% nilai total) |                                    |         |
| Source code publish di GitHub  | 5%                       | repo publik + README               | semua   |

| Item rubrik           | Bobot | Implementasi kita               | Pemilik |
|-----------------------|-------|---------------------------------|---------|
| Docker                | 15%   | docker-compose semua service    | C       |
| GUI (non-console)     | 5%    | Web dashboard                   | C       |
| Presentasi < 10 menit | 25%   | Video, semua anggota kontribusi | semua   |

Kalau semua kelar, ini full 100%.

## 4. Pembagian Tim (3 Orang)

Pembagian dibuat biar tiap orang punya 1 domain teknis yang jelas, minim tabrakan kode, dan tiap orang jadi "ahli" di bagiannya buat presentasi.

### Person A: Distributed Core & Leader Election

Fokus item paling berat dan paling mahal (HS = 20%).

- Implementasi Hirschberg-Sinclair election di Scheduler cluster (3 node).
- RPC antar node Scheduler (pesan election: probe, reply, terminate).
- Heartbeat, failure detection, dan re-election saat leader down.
- Logika "leader = otoritas commit slot".
- Presentasi: jelasin cara kerja HS + demo live failover.

### Person B: Backend Services, RPC & Messaging

Fokus alur data utama (REST 15% + RPC 20% + RabbitMQ 20%).

- Booking Service: endpoint REST ((`POST /appointments`), (`GET /appointments/{id}`), (`POST /appointments/{id}/confirm`)).
- Definisi proto gRPC + gRPC client ke Scheduler leader.
- RabbitMQ: publish event (`appointment_confirmed`) + Notification Worker (consumer).
- Logika retry kalau node yang dipanggil bukan leader (contek pola (`NOT_LEADER`) di demo).
- Presentasi: jelasin alur service-to-service + demo RabbitMQ async.

### Person C: Persistence, DevOps & Frontend

Fokus infrastruktur dan tampilan (DB 10% + Docker 15% + GUI 5% + LB 5%).

- Skema PostgreSQL + integrasi (SQLAlchemy/psycopg2) buat semua service.
- docker-compose: semua service + RabbitMQ + Postgres + nginx.
- Config nginx sebagai load balancer 2 replika Booking.
- Web dashboard: form booking, status appointment, status leader, log notifikasi.

- Presentasi: jelasin deployment Docker + walkthrough GUI + bukti data persist di DB.
- 

## 5. Kontrak Integrasi (WAJIB disepakati di awal)

Ini bagian paling sering bikin project tim berantakan. Sepakati semua interface ini di Hari 1-2, tulis di repo, baru kerja paralel. Setelah ini di-lock, tiap orang bisa kerja pakai mock tanpa nunggu yang lain.

### a. Proto gRPC Scheduler (dibuat B, dipakai A):

- `ReserveSlot(appointment_id, slot_time, service_id) -> {status, slot_id, leader_id}`
- `WhoIsLeader() -> {leader_id, leader_url, i_am_leader}`
- Error `NOT_LEADER` balikin info leader yang diketahui.

### b. Skema DB (dibuat C, dipakai B & A):

- `appointments(id, customer_name, service_id, slot_time, status, created_at)`
- `slots(id, slot_time, service_id, is_booked, booked_by)`
- `notifications(id, appointment_id, channel, status, sent_at)`

### c. Format pesan RabbitMQ (dibuat B):

- Event `appointment_confirmed`: `{appointment_id, customer_name, slot_time, channel, correlation_id}`

### d. Konvensi port & env (dibuat C di docker-compose):

- Booking 8000, Scheduler node 9000, RabbitMQ 5672/15672, Postgres 5432, nginx 80.
- 

## 6. Timeline (sesuaikan dengan deadline lo)

### Fase 0: Setup & Kontrak (Hari 1-2, semua hadir)

- Bikin repo GitHub, struktur folder, README skeleton.
- Sepakati arsitektur + semua kontrak di Bagian 5.
- C bikin skeleton docker-compose biar tiap orang bisa `docker compose up` bagian sendiri.

### Fase 1: Bangun Komponen Paralel (Minggu 1)

- A: HS election jalan di 3 node pakai data dummy (belum nyambung booking). Tes failover manual.

- B: Booking REST + gRPC client + RabbitMQ publish/worker, pakai DB mock atau Postgres lokal.
- C: Skema Postgres jadi, docker-compose semua service nyala, nginx LB jalan, skeleton GUI.

## Fase 2: Integrasi (Minggu 2)

- Sambungin Booking -> Scheduler leader -> commit slot ke DB.
- Worker konsumsi event -> tulis notifikasi ke DB.
- GUI nyambung ke Booking Service.
- Tes failover end-to-end: matiin leader pas lagi booking, pastikan sistem recover.

## Fase 3: Polish & Rekam (Minggu 3)

- Edge case: idempotency (double submit), leader down di tengah transaksi, node balik hidup.
  - README lengkap + diagram arsitektur + cara run.
  - Rekam video presentasi (< 10 menit, semua ngomong).
  - Publish repo, kumpulin link.
- 

## 7. Checklist Deliverable

- ☐ REST-API Booking Service jalan
  - ☐ gRPC stub Booking <-> Scheduler jalan
  - ☐ RabbitMQ pipeline notifikasi jalan
  - ☐ PostgreSQL nyimpen appointments + slots + notifications
  - ☐ Domain = appointment booking + notification (bukan payment)
  - ☐ Hirschberg-Sinclair election + demo failover berhasil
  - ☐ nginx load balancing 2 replika Booking
  - ☐ Semua service jalan via `docker compose up`
  - ☐ Web dashboard berfungsi
  - ☐ Repo publik di GitHub + README + diagram
  - ☐ Video presentasi < 10 menit, 3 orang kontribusi
  - ☐ Link dikumpulin sebelum deadline
- 

## 8. Tips Presentasi (25%, jangan disepelekan)

Bobotnya paling gede setelah implementasi. Syaratnya tiap anggota harus kontribusi.

Bagi jadi 3 segmen + intro/outro, total di bawah 10 menit:

- Intro bareng (~1 mnt): masalah + arsitektur high-level.

- Person A (~3 mnt): kenapa butuh leader, cara kerja Hirschberg-Sinclair, demo live matiin leader -> re-election.
- Person B (~3 mnt): alur REST -> gRPC -> leader -> RabbitMQ, demo booking + notifikasi async.
- Person C (~2 mnt): Docker deployment, GUI walkthrough, bukti data persist di DB + load balancing.

Pastikan suara dan kontribusi tiap orang kedengeran jelas. Rekam layar pas demo, jangan cuma slide.

---

## 9. Keputusan yang Perlu Lo Konfirmasi

1. **Hirschberg-Sinclair (20%) vs Ring (10%):** HS dua kali lipat poinnya tapi lebih susah. Rekomendasi: HS, karena Person A fokus penuh. Fallback aman: Ring kalau A gak pede.
2. **gRPC stub vs REST-RPC:** Rekomendasi gRPC biar item REST dan RPC kebedain jelas. Fallback: REST-RPC kayak demo (tetap sah dapat 20%).
3. **Stack:** Tetap Python (FastAPI/Flask + grpcio + pika + SQLAlchemy) biar bisa kontek banyak dari demo. GUI cukup HTML+JS atau React kecil.
4. **Opsional flex:** Kubernetes di atas Docker. Gak nambah poin rubrik (Docker udah cukup buat 15%), tapi impresif. Skip kalau waktu mepet.